MC458 - Projeto e Análise de Algoritmos I

Prof. Dr. Hilder Vitor Lima Pereira

Instituto de computação

UNICAMP

24 de março de 2026

1 Revisão

2 Divisão e conquista
- Multiplicação de inteiros

3 Perguntas ou comentários?

Revisão

Teorema Mestre

Sejam $a \geq 1$ e $b > 1$ constantes, seja $f(n)$ uma função e seja $T(n)$ a seguinte relação de recorrência

$$T(n) = aT(n/b) + f(n).$$

Teorema Mestre

Sejam $a \geq 1$ e $b > 1$ constantes, seja $f(n)$ uma função e seja $T(n)$ a seguinte relação de recorrência

$$T(n) = aT(n/b) + f(n).$$

Então $T(n)$ tem os seguintes limitantes assintóticos:

Teorema Mestre

Sejam $a \geq 1$ e $b > 1$ constantes, seja $f(n)$ uma função e seja $T(n)$ a seguinte relação de recorrência

$$T(n) = aT(n/b) + f(n).$$

Então $T(n)$ tem os seguintes limitantes assintóticos:

1. Se $f(n) \in O(n^{\log_b a - \epsilon})$ para alguma constante $\epsilon > 0$, então $T(n) \in \Theta(n^{\log_b a})$

Teorema Mestre

Sejam $a \geq 1$ e $b > 1$ constantes, seja $f(n)$ uma função e seja $T(n)$ a seguinte relação de recorrência

$$T(n) = aT(n/b) + f(n).$$

Então $T(n)$ tem os seguintes limitantes assintóticos:

1. Se $f(n) \in O(n^{\log_b a - \epsilon})$ para alguma constante $\epsilon > 0$, então $T(n) \in \Theta(n^{\log_b a})$
2. Se $f(n) \in \Theta(n^{\log_b a})$, então $T(n) \in \Theta(n^{\log_b a} \log n)$

Teorema Mestre

Sejam $a \geq 1$ e $b > 1$ constantes, seja $f(n)$ uma função e seja $T(n)$ a seguinte relação de recorrência

$$T(n) = aT(n/b) + f(n).$$

Então $T(n)$ tem os seguintes limitantes assintóticos:

1. Se $f(n) \in O(n^{\log_b a - \epsilon})$ para alguma constante $\epsilon > 0$, então $T(n) \in \Theta(n^{\log_b a})$
2. Se $f(n) \in \Theta(n^{\log_b a})$, então $T(n) \in \Theta(n^{\log_b a} \log n)$
3. Se $f(n) \in \Omega(n^{\log_b a + \epsilon})$ para alguma constante $\epsilon > 0$ e se $af(n/b) \leq cf(n)$ para alguma constante $c < 1$ e todo n suficientemente grande, então $T(n) \in \Theta(f(n))$

Divisão e conquista

Divisão e conquista:

Multiplicação de inteiros

O problema da multiplicação de inteiros

Problema

- Entrada: $x, y \in \mathbb{N}$ com no máximo n bits cada
- Saída: $z \in \mathbb{N}$ tal que $z = x \cdot y$

O problema da multiplicação de inteiros

Problema

- Entrada: $x, y \in \mathbb{N}$ com no máximo n bits cada
- Saída: $z \in \mathbb{N}$ tal que $z = x \cdot y$

- Obviamente, conforme n cresce, o produto deixa de ser uma operação básica

O problema da multiplicação de inteiros

Problema

- Entrada: $x, y \in \mathbb{N}$ com no máximo n bits cada
- Saída: $z \in \mathbb{N}$ tal que $z = x \cdot y$

- Obviamente, conforme n cresce, o produto deixa de ser uma operação básica
- Note que z tem até $2n$ bits

O problema da multiplicação de inteiros

Problema

- Entrada: $x, y \in \mathbb{N}$ com no máximo n bits cada
- Saída: $z \in \mathbb{N}$ tal que $z = x \cdot y$

- Obviamente, conforme n cresce, o produto deixa de ser uma operação básica
- Note que z tem até $2n$ bits

O problema da multiplicação de inteiros

Problema

- Entrada: $x, y \in \mathbb{N}$ com no máximo n bits cada
- Saída: $z \in \mathbb{N}$ tal que $z = x \cdot y$

- Obviamente, conforme n cresce, o produto deixa de ser uma operação básica
- Note que z tem até $2n$ bits

Você sabe calcular z usando $\Theta(n^2)$ operações básicas!

Multiplicação: Algoritmo trivial

O algoritmo que aprendemos na escola usa instâncias de tamanho $n-1$ na recursão...

- $a = \underbrace{a_0 \cdot 2^0 + a_1 \cdot 2^1 + ... + a_{n-1} \cdot 2^{n-1}}_{\hat{a}} + a_n \cdot 2^n$
- $b = \underbrace{b_0 \cdot 2^0 + b_1 \cdot 2^1 + ... + b_{n-1} \cdot 2^{n-1}}_{\hat{b}} + b_n \cdot 2^n$

Multiplicação: Algoritmo trivial

O algoritmo que aprendemos na escola usa instâncias de tamanho $n-1$ na recursão...

- $a = \underbrace{a_0 \cdot 2^0 + a_1 \cdot 2^1 + ... + a_{n-1} \cdot 2^{n-1}}_{\hat{a}} + a_n \cdot 2^n$
- $b = \underbrace{b_0 \cdot 2^0 + b_1 \cdot 2^1 + ... + b_{n-1} \cdot 2^{n-1}}_{\hat{b}} + b_n \cdot 2^n$
- Então,
$$a \cdot b = \hat{a}\hat{b} + \hat{a}b_n 2^n + a_n \hat{b} 2^n + a_n b_n 2^{2n}$$
- Por hipótese, sabemos calcular $x = \hat{a} \cdot \hat{b}$
- Como $a_n, b_n \in \{0, 1\}$, basta testar se eles são 1 e somar os valores correspondentes

Multiplicação: Algoritmo trivial

Número de operações básicas:

- Somar $x + a_n b_n 2^{2n}$ leva $O(1)$

Multiplicação: Algoritmo trivial

Número de operações básicas:

- Somar $x + a_n b_n 2^{2n}$ leva $O(1)$
- Somar inteiros de $2n$ bits leva $O(n)$
- Temos no máximo duas dessas somas

Multiplicação: Algoritmo trivial

Número de operações básicas:

- Somar $x + a_n b_n 2^{2n}$ leva $O(1)$
- Somar inteiros de $2n$ bits leva $O(n)$
- Temos no máximo duas dessas somas
- $T(n) = T(n-1) + 2 \cdot c_1 \cdot n + c_2$

Multiplicação: Algoritmo trivial

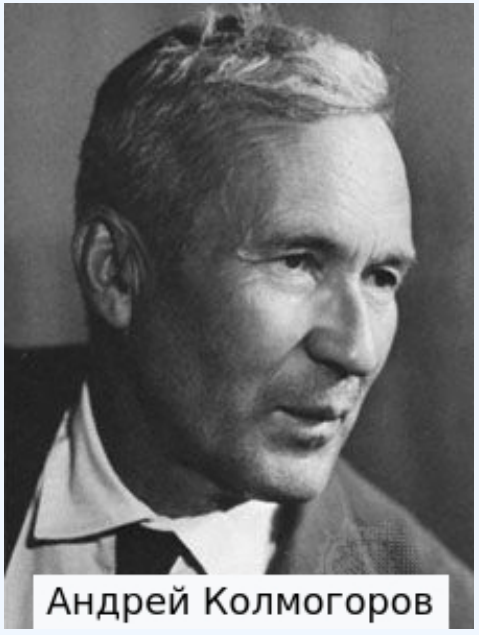
Número de operações básicas:

- Somar $x + a_n b_n 2^{2n}$ leva $O(1)$
- Somar inteiros de $2n$ bits leva $O(n)$
- Temos no máximo duas dessas somas
- $T(n) = T(n-1) + 2 \cdot c_1 \cdot n + c_2$

Assim, $T(n) \in O(n^2)$

Um pouco de história

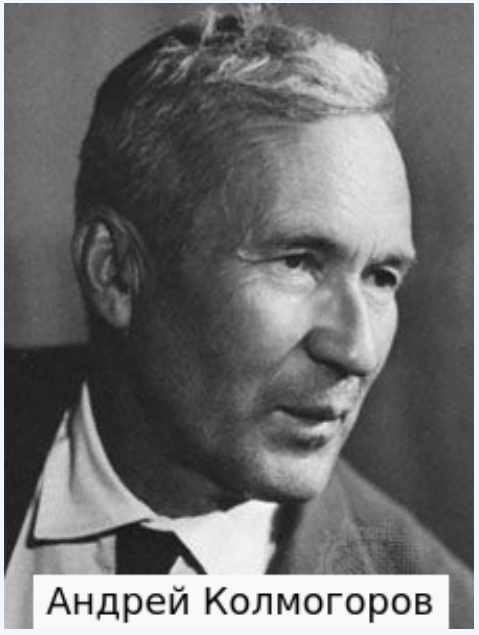
1960, num seminário em Moscou, União Soviética



Conjectura: multiplicar inteiros de n bits requer $\Omega(n^2)$ operações

Um pouco de história

1960, num seminário em Moscou, União Soviética



Conjectura: multiplicar inteiros de n bits requer $\Omega(n^2)$ operações

(Com 23 anos) Viu o seminário. Criou um algoritmo que leva $o(n^2)$ operações, mostrando que a conjectura era falsa.

Algoritmo de Karatsuba

Estratégia de divisão e conquista:

- Assuma que sabemos multiplicar inteiros com menos de n bits (indução forte)
- Escreva $u = u_E + u_D \cdot 2^{n/2}$ e $v = v_E + v_D \cdot 2^{n/2}$
- Note que
$$u \cdot v = u_E v_E + 2^{n/2} \cdot (u_E v_D + u_D v_E) + 2^n u_D v_D$$
- Então, sabemos multiplicar $u_i \cdot v_j$ (operandos têm $n/2$ bits)
- Essa foi a divisão... Como efetuamos a conquista?

Algoritmo de Karatsuba

Estratégia de divisão e conquista:

- Assuma que sabemos multiplicar inteiros com menos de n bits (indução forte)
- Escreva $u = u_E + u_D \cdot 2^{n/2}$ e $v = v_E + v_D \cdot 2^{n/2}$
- Note que
$$u \cdot v = u_E v_E + 2^{n/2} \cdot (u_E v_D + u_D v_E) + 2^n u_D v_D$$
- Então, sabemos multiplicar $u_i \cdot v_j$ (operandos têm $n/2$ bits)
- Essa foi a divisão... Como efetuamos a conquista?
- Usar recursão para calcular cada termo $u_i v_j$ nos dá
$$T(n) = 4T(n/2) + cn$$
- Ou seja, $T(n) \in \Theta(n^2)$
- Precisamos fazer menos que 4 chamadas recursivas...

Considere

$$u \cdot v = \underbrace{u_E v_E}_{x_0} + 2^{n/2} \cdot \underbrace{(u_E v_D + u_D v_E)}_{x_1} + 2^n \underbrace{u_D v_D}_{x_2}$$

Há alguma forma de calcular x_0, x_1 e x_2 com menos de 4 multiplicações?

Considere

$$u \cdot v = \underbrace{u_E v_E}_{x_0} + 2^{n/2} \cdot \underbrace{(u_E v_D + u_D v_E)}_{x_1} + 2^n \underbrace{u_D v_D}_{x_2}$$

Há alguma forma de calcular x_0, x_1 e x_2 com menos de 4 multiplicações?

Seja $y = (u_E + u_D) \cdot (v_E + v_D)$. Note que

$$y = u_E v_E + u_E v_D + u_D v_E + u_D v_D = x_0 + x_1 + x_2$$

Ou seja, o termo x_1, que exigia duas multiplicações, pode ser calculado como

$$x_1 = y - x_0 - x_2$$

Considere

$$u \cdot v = \underbrace{u_E v_E}_{x_0} + 2^{n/2} \cdot \underbrace{(u_E v_D + u_D v_E)}_{x_1} + 2^n \underbrace{u_D v_D}_{x_2}$$

Há alguma forma de calcular x_0, x_1 e x_2 com menos de 4 multiplicações?

Seja $y = (u_E + u_D) \cdot (v_E + v_D)$. Note que

$$y = u_E v_E + u_E v_D + u_D v_E + u_D v_D = x_0 + x_1 + x_2$$

Ou seja, o termo x_1, que exigia duas multiplicações, pode ser calculado como

$$x_1 = y - x_0 - x_2$$

Em suma,

- $x_0 \leftarrow u_E \cdot v_E$ . . . . . 1 mult
- $x_2 \leftarrow u_D \cdot v_D$ . . . . . 1 mult
- $y \leftarrow (u_E + u_D) \cdot (v_E + v_D)$ . . . . 1 mult
- $x_1 \leftarrow y - x_0 - x_2$ . . . . . 0 mult

Considere

$$u \cdot v = \underbrace{u_E v_E}_{x_0} + 2^{n/2} \cdot \underbrace{(u_E v_D + u_D v_E)}_{x_1} + 2^n \underbrace{u_D v_D}_{x_2}$$

Há alguma forma de calcular x_0, x_1 e x_2 com menos de 4 multiplicações?

Seja $y = (u_E + u_D) \cdot (v_E + v_D)$. Note que

$$y = u_E v_E + u_E v_D + u_D v_E + u_D v_D = x_0 + x_1 + x_2$$

Ou seja, o termo x_1, que exigia duas multiplicações, pode ser calculado como

$$x_1 = y - x_0 - x_2$$

Em suma,

- $x_0 \leftarrow u_E \cdot v_E$ . . . . . 1 mult
- $x_2 \leftarrow u_D \cdot v_D$ . . . . . 1 mult
- $y \leftarrow (u_E + u_D) \cdot (v_E + v_D)$ . . . . 1 mult
- $x_1 \leftarrow y - x_0 - x_2$ . . . . . 0 mult
- Total: **3** multiplicações de inteiros com $n/2$ bits

Algoritmo de Karatsuba

$\texttt{Karatsuba}(u, v)$

Input: $u, v \in \mathbb{N}$, ambos menores que 2^n
Output: $u \cdot v \in \mathbb{N}$

```
if n = 1 then
    return u · v                                  ▷ Mult. dois bits
else
    m ← ⌊n/2⌋
    (u_E, u_D) ← (u[0, ..., m], u[m + 1, ..., n − 1])
    (v_E, v_D) ← (v[0, ..., m], v[m + 1, ..., n − 1])
    y_0 ← Add(u_E, u_D)
    y_1 ← Add(v_E, v_D)
    y ← Karatsuba(y_0, y_1)
    x_0 ← Karatsuba(u_E, v_E)
    x_2 ← Karatsuba(u_D, v_D)
    x_1 ← Sub(Sub(y, x_0), x_2)
    p ← Add(x_0, Add(Shift(x_1, m), Shift(x_2, n)))
    return p
```

Karatsuba: Complexidade computacional

Como `Add` e `Shift` efetuam $O(n)$ operações básicas, a complexidade do Karatsuba é

$$T(n) = \begin{cases} c & \text{se } n = 1 \\ 3T(\lceil n/2 \rceil) + \hat{c}n & \text{se } n \geq 2 \end{cases}$$

onde c e $\hat{c}$ são constantes

Karatsuba: Complexidade computacional

Como `Add` e `Shift` efetuam $O(n)$ operações básicas, a complexidade do Karatsuba é

$$T(n) = \begin{cases} c & \text{se } n = 1 \\ 3T(\lceil n/2 \rceil) + \hat{c}n & \text{se } n \geq 2 \end{cases}$$

onde c e $\hat{c}$ são constantes

Pelo Teorema Mestre, temos

$$T(n) \in \Theta\left(n^{\log_2(3)}\right)$$

Note que $\log_2(3) \approx 1,5849 < 2$, ou seja, temos um algoritmo assintoticamente mais rápido do que o algoritmo quadrático.

Karatsuba: Discussão

Percebam que tanto a parte de *divisão* quanto a de *conquista* não são triviais:

- um projeto ingênuo geraria 4 chamadas recursivas, nos dando um algoritmo quadrático
- foi preciso pensar sobre as propriedades algébricas dos valores envolvidos
- isso nos permitiu melhorar a etapa de divisão (3 recursões) e obter uma nova etapa de conquista

Karatsuba: Discussão

Percebam que tanto a parte de *divisão* quanto a de *conquista* não são triviais:

- um projeto ingênuo geraria 4 chamadas recursivas, nos dando um algoritmo quadrático
- foi preciso pensar sobre as propriedades algébricas dos valores envolvidos
- isso nos permitiu melhorar a etapa de divisão (3 recursões) e obter uma nova etapa de conquista

Muitas vezes, é preciso encontrar uma forma inteligente de usar divisão e conquista

Perguntas ou comentários?